

PROJET INDIVIDUEL

Développement Mobile Android ShopMa

Application e-commerce marketplace

1. Problématique

Vous êtes chargé de développer ShopMa, une application mobile Android de type marketplace e-commerce. L'application permet à un utilisateur de parcourir un catalogue de produits organisé par catégories, de les ajouter à un panier, de passer des commandes, et de suivre ses achats. Le catalogue est chargé depuis une API REST externe (FakeStoreAPI).

L'application doit couvrir les fonctionnalités suivantes :

- Un écran d'authentification avec validation de formulaire et mémorisation de l'identifiant (SharedPreferences).
- Un dashboard organisé par catégories de produits (6 cartes cliquables) avec une Toolbar contenant les icônes Panier et Profil.
- Un catalogue de produits chargé depuis FakeStoreAPI via Retrofit, filtrable par catégorie.
- Un panier d'achat persistant (SQLite) accessible depuis la Toolbar, avec gestion des quantités et calcul du total.
- Un historique des commandes stocké localement (SQLite), accessible depuis l'écran de profil.
- Une carte Google Maps affichant les points de retrait.
- Un écran de profil avec prise de photo (caméra) et sauvegarde des préférences.
- Des notifications locales (confirmation de commande).

API utilisée : FakeStoreAPI

Base URL : <https://fakestoreapi.com>

Endpoints utiles :

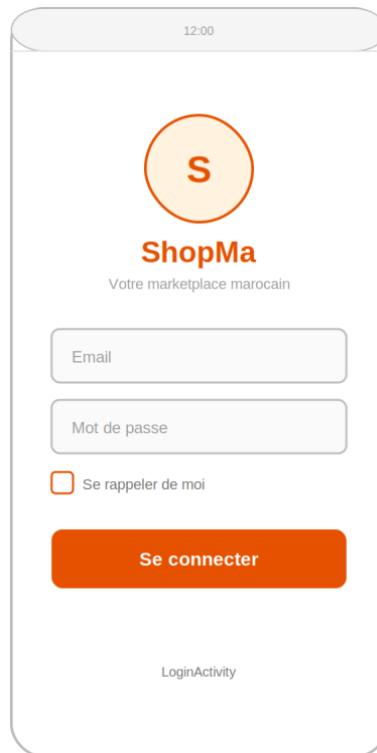
- GET /products : liste de tous les produits (id, title, price, description, category, image)
- GET /products/categories : liste des catégories disponibles
- GET /products/category/{nom} : produits filtrés par catégorie
- GET /products/{id} : un produit par son identifiant

Documentation complète : <https://fakestoreapi.com/docs>

2. Maquettes des écrans

Les wireframes ci-dessous décrivent l'interface attendue pour chaque écran. Vous devez respecter la structure générale (placement des éléments, navigation) mais vous êtes libres sur les choix esthétiques (couleurs, polices, icônes).

2.1 Connexion (LoginActivity)



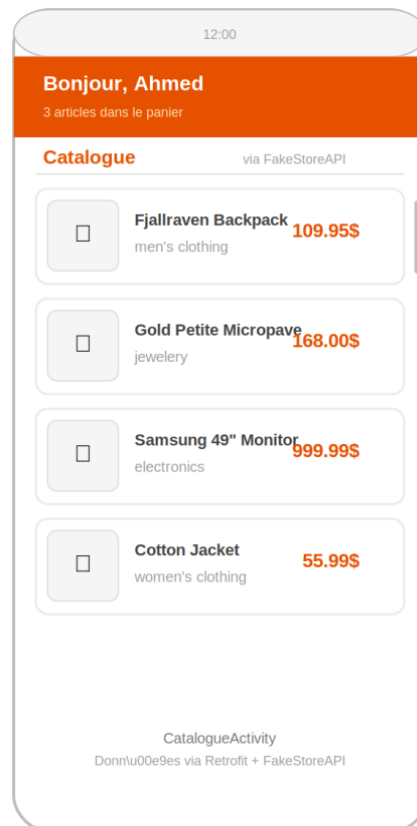
Logo, titre, deux champs (email + mot de passe), case « Se rappeler de moi », bouton de connexion. Validation : champs non vides, mot de passe ≥ 6 caractères. Identifiants vérifiables en dur. Si la case est cochée, l'email est sauvegardé en SharedPreferences et restauré au prochain lancement.

2.2 Dashboard (AccueilActivity)



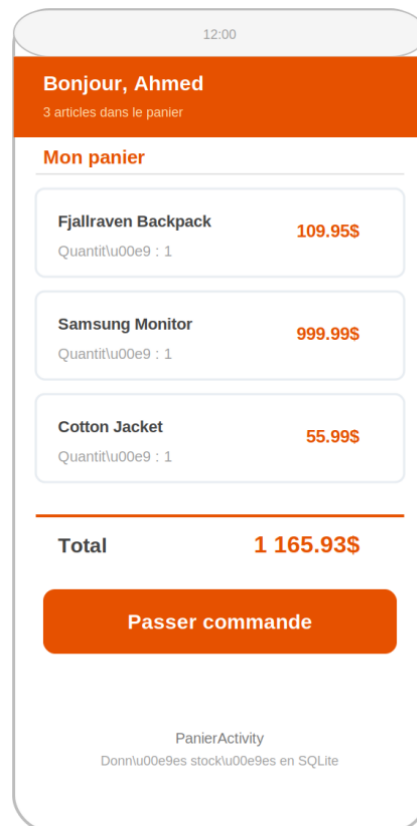
La Toolbar en haut contient le titre de l'application et deux icônes : le panier (avec un badge indiquant le nombre d'articles) et le profil. En dessous, un message de bienvenue, puis une grille 2x3 de cartes représentant les catégories de produits et les points de retrait. Chaque carte de catégorie lance CatalogueActivity avec la catégorie en extra (ou sans extra pour « Tous les produits »).

2.3 Catalogue (CatalogueActivity)



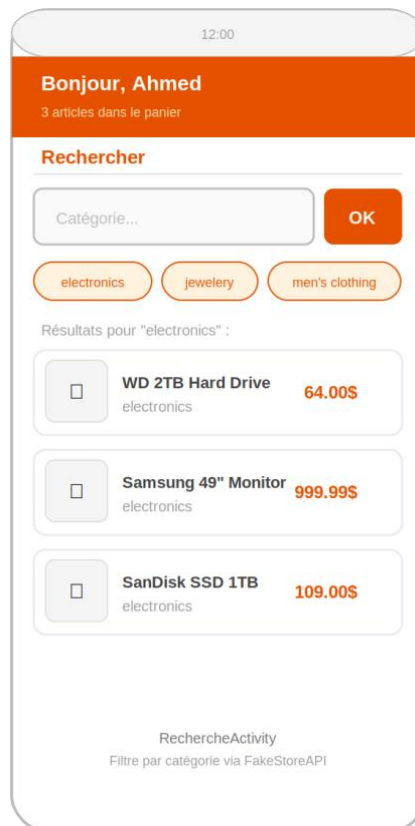
Le HeaderFragment en haut (nom de l'utilisateur + nombre d'articles dans le panier). En dessous, une ListView de produits chargés via Retrofit depuis FakeStoreAPI. Si une catégorie est passée en extra, l'écran affiche uniquement les produits de cette catégorie (endpoint `/products/category/{nom}`). Sinon, tous les produits (endpoint `/products`). Chaque item affiche le titre, la catégorie, le prix. Un Adapter personnalisé (BaseAdapter) est attendu. Au clic sur un produit, l'utilisateur peut l'ajouter au panier (insertion dans SQLite).

2.4 Panier (PanierActivity)



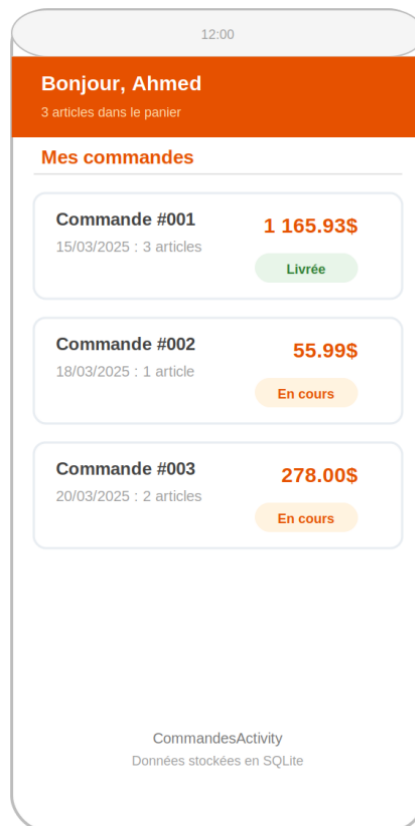
Accessible depuis l'icône panier de la Toolbar. Le HeaderFragment en haut, puis un ListView d'articles ajoutés, stockés en SQLite. Chaque item montre le nom, la quantité, le prix. Le total est calculé et affiché en bas. Le bouton « Passer commande » transfère les articles vers la table commandes (SQLite), vide le panier, et envoie une notification locale de confirmation.

2.5 Recherche (RechercheActivity)



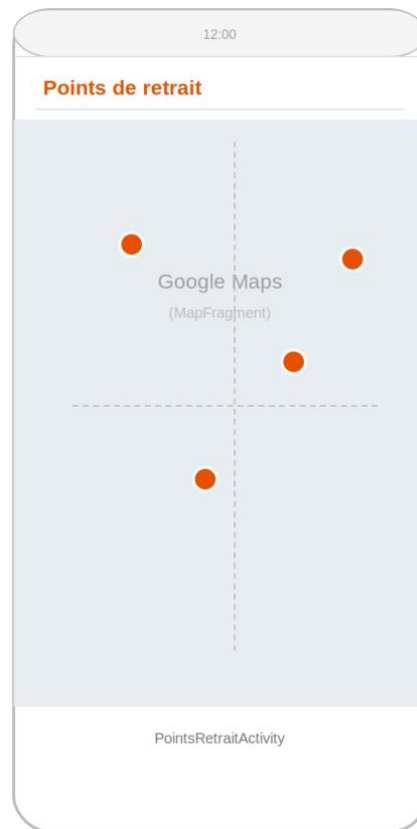
Le HeaderFragment en haut, puis un champ de saisie avec un bouton OK, des suggestions de catégories (electronics, jewelery, men's clothing, women's clothing), et une ListView affichant les résultats. La recherche interroge FakeStoreAPI par catégorie (GET /products/category/{nom}). L'Adapter peut être le même que celui du catalogue.

2.6 Mes commandes (CommandesActivity)



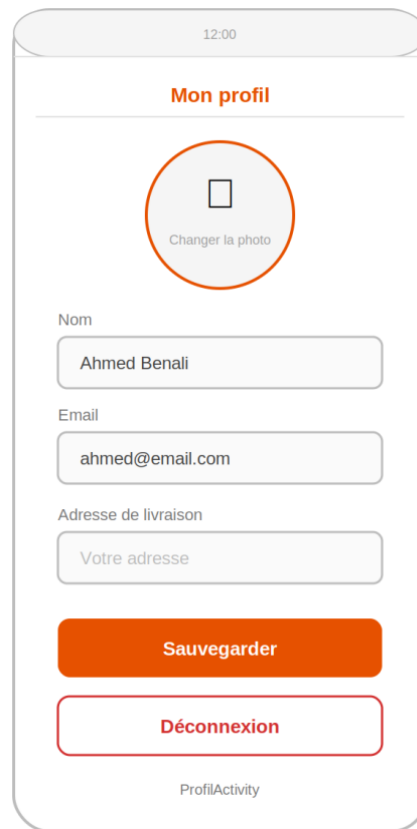
Accessible depuis l'écran de profil (bouton « Historique des commandes »). Le HeaderFragment en haut, puis un ListView de commandes passées, stockées dans SQLite. Chaque item montre le numéro, la date, le nombre d'articles, le montant total, et un badge de statut (En cours ou Livrée). Un Adapter personnalisé est attendu pour le badge coloré.

2.7 Points de retrait (PointsRetraitActivity)



Un titre en haut et un `SupportMapFragment` (Google Maps SDK) qui occupe le reste de l'écran. Au moins 4 marqueurs sur des points de retrait au Maroc (coordonnées de votre choix). Chaque marqueur a un titre et un snippet (adresse). Caméra centrée sur le Maroc avec un zoom adapté.

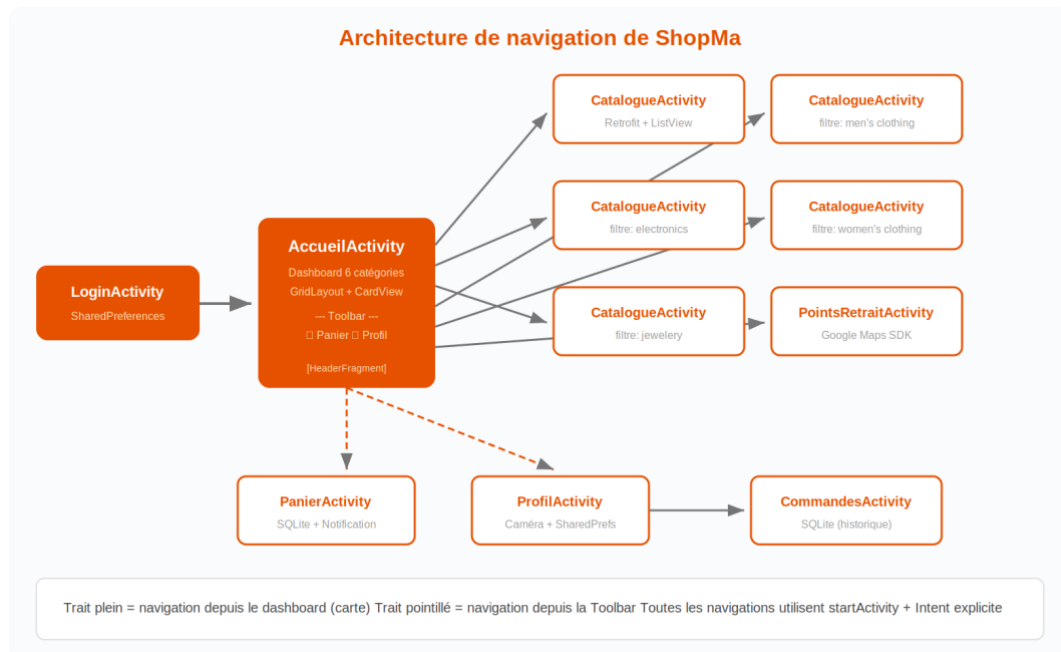
2.8 Profil (ProfilActivity)



Accessible depuis l'icône profil de la Toolbar. Photo de l'utilisateur (prise avec la caméra via ACTION_IMAGE_CAPTURE), nom, email, adresse de livraison. Informations sauvegardées en SharedPreferences. Un bouton « Historique des commandes » qui ouvre CommandesActivity. Un bouton « Déconnexion » qui ramène au login.

3. Architecture de navigation

Le schéma ci-dessous montre le flux de navigation complet. Les traits pleins représentent les navigations depuis les cartes du dashboard. Les traits pointillés représentent les navigations depuis la Toolbar (panier et profil).



L'application comporte 8 Activities au total. Le HeaderFragment est réutilisable et doit apparaître dans au moins 4 écrans sans duplication de code. Toutes les navigations utilisent des Intents explicites. Le nom de l'utilisateur est passé en extra à chaque Activity qui intègre le HeaderFragment.

4. Contraintes techniques

Le projet doit obligatoirement utiliser les technologies suivantes :

Concept	Utilisation attendue
Layouts	LinearLayout, GridLayout (dashboard), ConstraintLayout ou RelativeLayout. Ressources externalisées (strings.xml, colors.xml, dimens.xml).
Toolbar	Barre d'actions avec icônes panier (avec badge) et profil.
findViewById	Récupération des vues dans toutes les Activities.
ListView + BaseAdapter	Catalogue, panier, commandes, recherche. Au moins un Adapter personnalisé avec getView() et recyclage convertView.
Fragment	HeaderFragment réutilisable dans au moins 4 Activities.
SharedPreferences	Email (Se rappeler de moi) et informations du profil.
SQLite	Deux tables minimum : panier et commandes. DatabaseHelper avec onCreate, insertion, lecture, suppression.
Intents explicites	Navigation entre toutes les Activities avec passage d'extras.
Intents implicites	Au moins un : partager un produit par email ou appeler le service client.
Retrofit + Gson	Chargement du catalogue. Au moins deux endpoints (liste complète + filtre par catégorie).
Google Maps SDK	MapFragment avec au moins 4 marqueurs.
Caméra	Photo de profil via ACTION_IMAGE_CAPTURE.
Notifications locales	Confirmation de commande. Canal de notification obligatoire.
Permissions runtime	CAMERA et ACCESS_FINE_LOCATION avec checkSelfPermission/requestPermissions.

5. Spécifications de l'API FakeStoreAPI

FakeStoreAPI est gratuite, publique, et ne nécessite aucune authentification.

5.1 Structure JSON d'un produit

```
{
  "id": 1,
  "title": "Fjallraven - Foldsack No. 1 Backpack",
  "price": 109.95,
  "description": "Your perfect pack for everyday use...",
  "category": "men's clothing",
  "image": "https://fakestoreapi.com/img/81fPKd-2AYL._AC_SL1500_.jpg",
  "rating": { "rate": 3.9, "count": 120 }
}
```

Le modèle Java (classe Product) doit avoir les attributs int id, String title, double price, String description, String category, String image. Les noms doivent correspondre exactement aux clés JSON.

5.2 Endpoints à utiliser

URL	Retourne
GET /products	Liste de 20 produits
GET /products/categories	["electronics","jewelery","men's clothing","women's clothing"]
GET /products/category/{nom}	Produits filtrés par catégorie
GET /products/{id}	Un seul produit par son identifiant

5.3 Interface Retrofit attendue

Votre interface ApiService doit comporter au minimum :

- Une méthode annotée @GET("products") retournant Call<List<Product>>
- Une méthode annotée @GET("products/categories") retournant Call<List<String>>
- Une méthode annotée @GET("products/category/{cat}") avec @Path("cat") retournant Call<List<Product>>

6. Structure de la base de données locale

Votre application doit comporter une base SQLite avec au minimum deux tables :

6.1 Table panier

Colonne	Type	Description
id	INTEGER PK	Identifiant auto-incrémenté
product_id	INTEGER	Identifiant du produit (venant de l'API)
title	TEXT	Nom du produit
price	REAL	Prix unitaire
quantity	INTEGER	Quantité dans le panier

6.2 Table commandes

Colonne	Type	Description
id	INTEGER PK	Identifiant auto-incrémenté
date	TEXT	Date de la commande (format dd/MM/yyyy)
nb_articles	INTEGER	Nombre total d'articles
montant_total	REAL	Montant total de la commande
statut	TEXT	Statut : en_cours ou livree

Le DatabaseHelper doit implémenter : création des tables, ajout au panier, lecture du panier, suppression du panier (après commande), ajout d'une commande, lecture de l'historique, comptage des articles dans le panier (pour le badge Toolbar).

7. Bonus : backend personnalisé

L'utilisation de FakeStoreAPI est le minimum attendu. Si vous souhaitez aller plus loin, vous pouvez créer votre propre backend et l'utiliser à la place de (ou en complément de) FakeStoreAPI. Ce travail supplémentaire sera comptabilisé comme un bonus sur la note globale du projet.

Plusieurs options sont possibles :

- **mockapi.io** : un service en ligne gratuit qui permet de créer des endpoints REST personnalisés avec vos propres données (produits marocains, prix en MAD, catégories adaptées). Aucune installation nécessaire.
- **json-server** : un outil Node.js qui transforme un fichier JSON en API REST complète (GET, POST, PUT, DELETE). Vous définissez vos données dans un fichier db.json et le serveur démarre en une commande.
- **Backend complet** : si vous maîtrisez un framework web (Spring Boot, Express.js, Django, Laravel...), vous pouvez créer un vrai backend avec authentification, gestion des commandes côté serveur, et endpoints sur mesure.

Pour être pris en compte, le bonus doit être fonctionnel et documenté : indiquez dans un fichier README.md l'URL du backend, la technologie utilisée, et les endpoints disponibles. Le backend doit être accessible lors de la correction (soit déployé en ligne, soit avec des instructions de démarrage local claires).

8. Consignes générales

1. Le projet est strictement individuel. Tout plagiat entraînera la note de 0.
2. Le langage est Java. Kotlin n'est pas accepté.
3. Toutes les chaînes dans strings.xml, toutes les couleurs dans colors.xml, toutes les dimensions dans dimens.xml.
4. L'application doit fonctionner sans crash sur un émulateur API 24+ ou un téléphone physique.
5. Les permissions runtime doivent être gérées : pas de crash si l'utilisateur refuse.
6. Le HeaderFragment doit être utilisé dans au moins 4 Activities sans duplication de code.
7. Gestion des erreurs réseau : si l'API est inaccessible, un message clair au lieu d'un crash.
8. Liberté sur le design visuel tant que la structure des écrans respecte les wireframes.
9. L'utilisation de bibliothèques supplémentaires (Picasso, Glide) est autorisée mais pas obligatoire.
10. Le livrable est le projet Android Studio complet compressé en .zip (pas un APK seul).

Ce projet synthétise l'ensemble des concepts abordés dans les 7 ateliers. Relisez vos ateliers en cas de doute. La documentation officielle Android (developer.android.com) et la documentation FakeStoreAPI (fakestoreapi.com/docs) sont vos meilleures ressources.